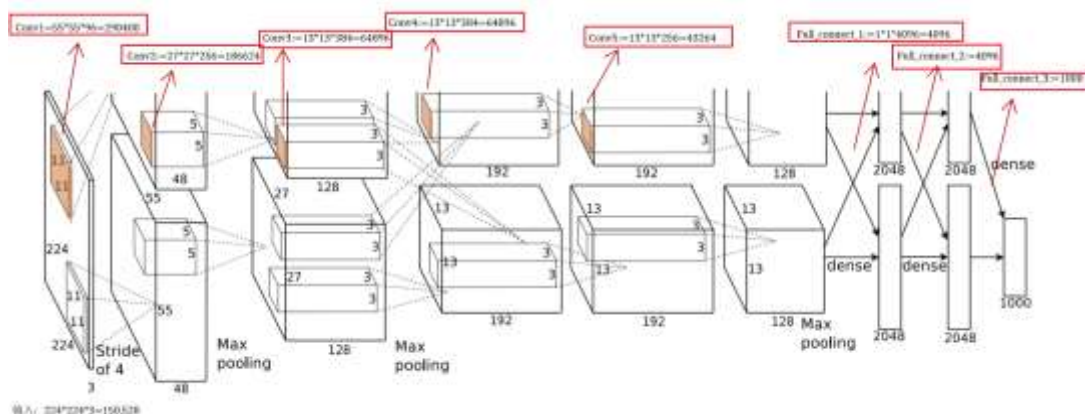


## 01. 如果不考虑偏置b，现在整个LeNet网络中参数w的数量是多少？

$6*5*5$  (conv1) +  $16*6*5*5$  (conv2) +  $400*120$  (full\_1) +  $120*84$  (full\_2)  
+  $84*10$  (full\_3) = 61470 个参数

## 02. AlexNet原论文中，如图红线中标注处有误，请找出错误，并结合所学知识进行更正，并说明问题出在哪里？



问题出在 253440 计算错误，应该为 290400，其他 4 个卷积层+3 个全连接层计算没错

Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

另外陶老师有个问题，我看网上有资料对第一个最大池化的位置放在了 Conv2 (kernel size=5\*5 的) 前面，这个怎么理解，论文图里面意思好像是“第一个最大池池化的位置在 Conv2 (kernel size=5\*5 的) 后面”能讲讲吗，是论文图画错了吗？



### 03. 视频中VGG网络实现，查看每层输出时，为什么层的数量总会少一些？

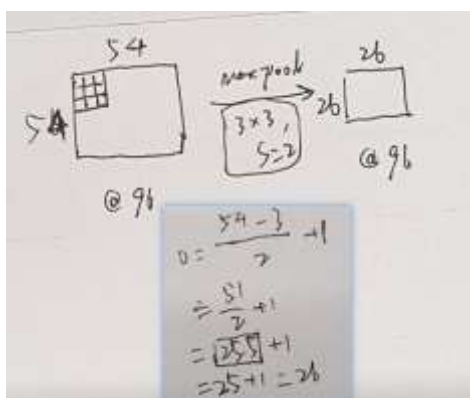
查看输出形状 `print` 的是 `net_11`，这个函数是 `vgg` 网络传递给他的，所以核心原因是查看的是 `vgg` 网络的每一层输出，而看不到 `block` 内部的层。

```
def vgg_conv_arch(in_channels=1):
    conv_bkls = []
    in_channels = in_channels
    # 卷积层
    for (num_convs, out_channels) in conv_arch:
        conv_bkls.append(vgg_block(num_convs, in_channels, out_channels))
        in_channels = out_channels
    return nn.Sequential(
        *conv_bkls,
        nn.Flatten(),
        # 全连接层
        nn.Linear(out_channels * 7 * 7, 4096),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(4096, 4096),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(4096, 10))
```

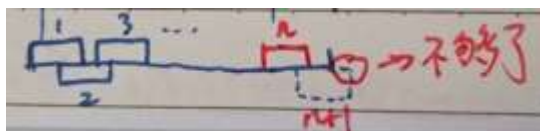
### 04. MNIST数据集可以使用11层或更多层的VGG网络么？为什么？

MNIST 是  $28 \times 28$  的灰度图，比 ImageNet 的  $224 \times 224$  小了 64 倍，假如 MNIST 使用了 VGG11，要经历五次最大池化（`nn.MaxPool2d`），那么特征图尺寸应该是  $1 \times 1$ ，相当于把图“压扁了”，5 层卷积明显对于 MNIST 是过度拟合了，不建议用。

### 05. 观察视频中（037NiN网络2-结构分析与代码实现）定义的NiN网络结构，每层输出的形状为什么会由54变为26而不是27？



因为向下取整（`floor`）



在 PyTorch 的设计逻辑中：卷积核（以及池化运算）从左到右、从上到下滑动时，卷积核必须完全覆盖输入区域（不能只覆盖部分像素），所以最后多出的像素被丢弃，这个就是向下取整（`Floor`）。从 AlexNet 开始，CNN 框架（TensorFlow/PyTorch）都统一

用向下取整。

## 06. NiN网络中是没有全连接层的，这样做有什么好处？

先看 NiN 网络的策略是

NiN 最后一步输出是

---

```
ninblock(384,10,3,1,1), #一个block 里面包含一个 3x3 卷积 + 两个1x1 卷积 → 输出10 通道
nn.AdaptiveAvgPool2d((1,1)), # 全局平均池化 → 10 个值
nn.Flatten() # 撑开变成10 维向量
```

---

输出形状：torch.Size([1, 10])（10 个分类得分）

NiN 策略最后一步的参数量： $384 \times 10 = 3,840 \approx 3.8k$

如果选择了类似 VGG16 的方案，也就是后面有全连接层，参数量就会非常恐怖，应该是  $384 * (5 * 5) * 4096 = 39,321,600 \approx 39M$  参数，所以好处是参数少了，计算量很小。

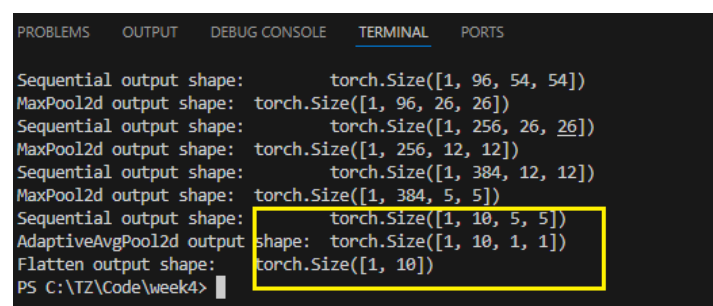


The screenshot shows a snippet of VGG16 code with handwritten annotations and parameter calculations. A red arrow points to the 'out\_channels' parameter in the first linear layer, which is 512. The calculations are as follows:

- For the first linear layer:  $512 * 7 * 7 * 4096 = 102,760,448$
- For the second linear layer:  $4096 * 4096 = 16,777,216$
- For the third linear layer:  $4096 * 10 = 40,960$

## 07. 如果用视频中（037NiN网络2-结构分析与代码实现）定义出来的网络进行图像的分类任务，应当是几类别？

10 类别，查看形状即可得知 `ninblock(384,10,3,1,1)` 这行代码里面的第二个参数“10”决定了类别数



The screenshot shows the terminal output of a PyTorch model's layers. The output shapes are as follows:

- Sequential output shape: torch.Size([1, 96, 54, 54])
- MaxPool2d output shape: torch.Size([1, 96, 26, 26])
- Sequential output shape: torch.Size([1, 256, 26, 26])
- MaxPool2d output shape: torch.Size([1, 256, 12, 12])
- Sequential output shape: torch.Size([1, 384, 12, 12])
- MaxPool2d output shape: torch.Size([1, 384, 5, 5])
- Sequential output shape: torch.Size([1, 10, 5, 5])
- AdaptiveAvgPool2d output shape: torch.Size([1, 10, 1, 1])
- Flatten output shape: torch.Size([1, 10])

## 08.使用Python搭建AlexNet模型

```
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl

# 设置中文字体（避免乱码）
mpl.rcParams['font.sans-serif'] = ['SimHei']
mpl.rcParams['axes.unicode_minus'] = False

# 检查 GPU 可用性并设置设备
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"使用设备: {device}")

lr = 0.001
batch_size = 128
epochs = 16

# 加载数据
train_X, test_X, train_y, test_y = np.load('mnist.npy',
allow_pickle=True)

# 转换为 tensor
x_train = torch.tensor(train_X, dtype=torch.float32)
y_train = torch.tensor(train_y)
x_test = torch.tensor(test_X, dtype=torch.float32)
y_test = torch.tensor(test_y)

#创建 Dataset
train_ds = TensorDataset(x_train, y_train)
test_ds = TensorDataset(x_test, y_test)
train_dl = DataLoader(train_ds, batch_size=batch_size, drop_last=True)
test_dl = DataLoader(test_ds, batch_size=batch_size, shuffle=False)

#AlexNet 模型构建
AlexNet = nn.Sequential(
    nn.Conv2d(1, 96, kernel_size=5, stride=1, padding=2),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),

    nn.Conv2d(96, 256, kernel_size=5, padding=2),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
```

```

nn.Conv2d(256, 384, kernel_size=3, padding=1),
nn.ReLU(),
nn.Conv2d(384, 384, kernel_size=3, padding=1),
nn.ReLU(),
nn.Conv2d(384, 256, kernel_size=3, padding=1),
nn.ReLU(),
nn.MaxPool2d(kernel_size=3, stride=2),

nn.Flatten(),
nn.Linear(1024, 4096),
nn.ReLU(),
nn.Dropout(p=0.5),
nn.Linear(4096, 4096),
nn.ReLU(),
nn.Dropout(p=0.5),
nn.Linear(4096, 10)
).to(device)

import torch.nn.functional as F
loss_func = F.cross_entropy
opt = torch.optim.SGD(AlexNet.parameters(), lr)

# 初始化指标记录列表
train_losses = []
train_accs = []
test_accs = []

# 训练循环
for epoch in range(epochs):
    # 1. 训练阶段
    AlexNet.train()
    epoch_loss = 0.0
    correct = 0
    total = 0

    for xb, yb in train_dl:
        xb = xb.to(device)
        yb = yb.to(device)

        xb = xb.reshape(-1, 1, 28, 28)
        pred = AlexNet(xb)
        loss = loss_func(pred, yb)

```

```

    opt.zero_grad()
    loss.backward()
    opt.step()

    epoch_loss += loss.item()
    _, predicted = torch.max(pred, 1)
    total += yb.size(0)
    correct += (predicted == yb).sum().item()

#计算整个 epoch 的平均损失和准确率
avg_train_loss = epoch_loss / len(train_dl)
train_acc = correct / total
train_losses.append(avg_train_loss)
train_accs.append(train_acc)

# 2. 测试阶段
AlexNet.eval()
test_correct = 0
test_total = 0
with torch.no_grad():
    for xb_test, yb_test in test_dl:
        xb_test = xb_test.to(device)
        yb_test = yb_test.to(device)

        xb_test = xb_test.reshape(-1, 1, 28, 28)
        pred_test = AlexNet(xb_test)
        _, predicted_test = torch.max(pred_test, 1)
        test_total += yb_test.size(0)
        test_correct += (predicted_test == yb_test).sum().item()

test_acc = test_correct / test_total
test_accs.append(test_acc)

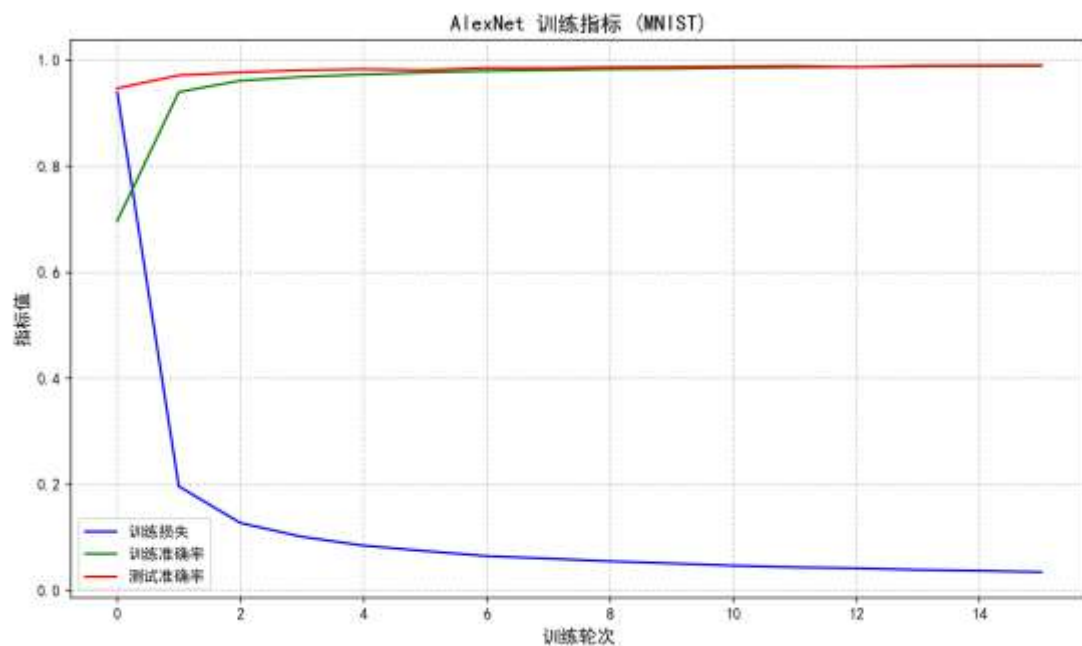
#打印整个 epoch 的指标
print(f"Epoch {epoch+1}/{epochs}, "
      f"Train Loss: {avg_train_loss:.4f}, "
      f"Train Acc: {train_acc:.4%}, "
      f"Test Acc: {test_acc:.4%}")

plt.figure(figsize=(10, 6))
plt.plot(train_losses, 'b-', label='训练损失')
plt.plot(train_accs, 'g-', label='训练准确率')
plt.plot(test_accs, 'r-', label='测试准确率')
plt.title('AlexNet 训练指标 (MNIST)', fontsize=14)

```

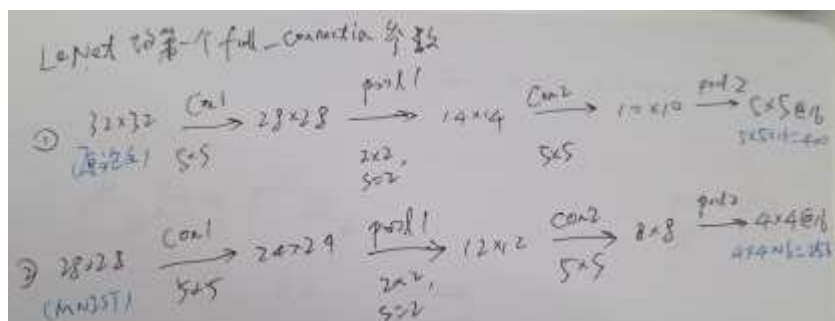
```
plt.xlabel('训练轮次', fontsize=12)
plt.ylabel('指标值', fontsize=12)
plt.legend(fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig('alexnet_mnist_metrics.png', dpi=150)
plt.close()

print("\n✅ 训练完成! 图表已保存为 'alexnet_mnist_metrics.png'")
print(f"最终测试准确率: {test_accs[-1]:.4%}")
```



### 09.搭建LeNet, 使用MNIST数据集进行手写数字识别的分类训练。

要求: 注意原论文中的样本数据与现有MNIST数据集中样本的大小区别, 调整自己所设计的LeNet网络结构中第一个全连接层的参数形状。





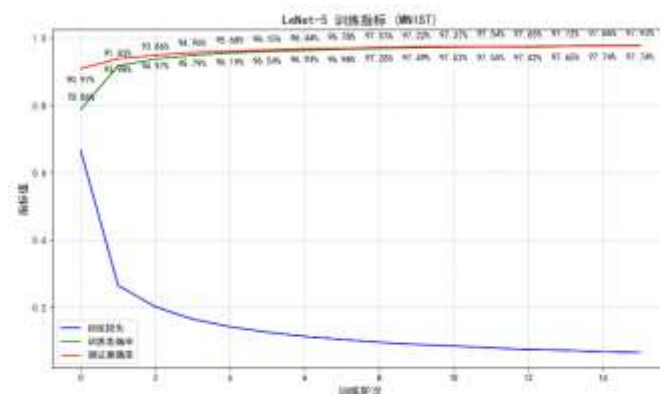
```

# LeNet-5 训练代码
def train():
    # 训练数据
    train_data = torchvision.datasets.MNIST(
        root='./mnist', train=True, transform=torchvision.transforms.Compose([
            torchvision.transforms.ToTensor(),
            torchvision.transforms.Normalize((0.1307,), (0.3081,))
        ]))
    # 测试数据
    test_data = torchvision.datasets.MNIST(
        root='./mnist', train=False, transform=torchvision.transforms.Compose([
            torchvision.transforms.ToTensor(),
            torchvision.transforms.Normalize((0.1307,), (0.3081,))
        ]))
    # 训练模型
    model = nn.Sequential(
        nn.Conv2d(1, 6, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),
        nn.Conv2d(6, 16, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),
        nn.Flatten(),
        nn.Linear(128, 120),
        nn.ReLU(),
        nn.Linear(120, 84),
        nn.ReLU(),
        nn.Linear(84, 10)
    )
    # 训练过程
    optimizer = optim.Adam(model.parameters())
    data_loader = torch.utils.data.DataLoader(train_data)
    test_loader = torch.utils.data.DataLoader(test_data)

    # 训练过程
    for epoch in range(1, 15):
        train_loss, train_acc, test_acc = train_epoch(model, optimizer, data_loader, test_loader)
        print(f'Epoch {epoch}: Train Loss: {train_loss}, Train Acc: {train_acc}, Test Acc: {test_acc}')

    # 保存模型
    torch.save(model.state_dict(), 'lenet5.pth')

```



10.将本周理论部分视频中所有代码尝试复现，并将练习的代码文件提交给小助理。